

Enterprise AI Knowledge & Action Platform

Project Master Documentation

Purpose: A practical, production-oriented AI engineering project combining enterprise RAG, hybrid search, agent/tool orchestration, asynchronous processing, reliability, security, testing, observability and product engineering.

Prepared for: Software Engineering Portfolio / Interview Preparation

Technology direction: Python, FastAPI, PostgreSQL + pgvector, Redis, React/Next.js, LLM APIs, Docker, GitHub Actions

1. Project Overview

Project name: Enterprise AI Knowledge & Action Platform

The platform is a private AI assistant for an organization. It combines company documents, structured business data and authorized tools so that users can ask questions and complete controlled actions through one interface.

The key engineering goal is not to build another generic chatbot. The system should demonstrate how an AI application is designed as a reliable software system: retrieval quality, permissions, validation, asynchronous jobs, caching, failure handling, observability and measurable evaluation.

2. Project Idea

Build a web-based enterprise assistant that can ingest internal documents such as policies, handbooks and technical manuals, retrieve relevant information using hybrid search, generate answers with citations, and later call approved business tools.

Example

An HR user asks: “What is the maternity leave policy?” The system retrieves the relevant policy sections and produces a cited answer.

A more advanced request could be: “Can employee 1024 take leave next Monday?” The agent may need to combine employee data, company policy and a holiday/calendar tool before answering.

Core principle

AI is one component of the platform—not the entire product. Deterministic software controls authentication, authorization, data access, tool permissions, state, validation and failure recovery.

3. End Users

- **HR teams:** Search policies, benefits, leave rules and employee-related information.
- **IT/support teams:** Search technical documentation and troubleshooting knowledge.
- **Employees:** Ask authorized questions about internal policies and procedures.
- **Administrators:** Manage users, documents, permissions, processing jobs and audit information.

For the MVP, select one concrete persona and workflow rather than attempting to serve every department.

4. Problem Statement

- Enterprise information is distributed across PDFs, Markdown files, databases and internal tools.
- Keyword-only search misses semantic matches; vector-only search can miss exact terms and identifiers.
- LLMs can hallucinate when they do not have reliable context.
- Business actions require authorization and deterministic validation.
- Large document ingestion can be slow and should not block API requests.
- Production AI systems need monitoring, evaluation, retries, security and cost controls.

5. Existing Solutions

The project should be positioned against categories rather than claiming that it is the first enterprise AI assistant. Existing products and frameworks already provide document search, enterprise connectors, RAG and agent capabilities.

- Enterprise AI assistants: strong integrations and polished user experience, but often optimized for broad commercial use.
- Cloud AI search/RAG services: provide managed retrieval infrastructure but can introduce platform dependence.
- Open-source RAG frameworks: accelerate implementation but do not automatically solve product requirements, security or reliability.
- Internal company search: good for known documents but often lacks natural-language reasoning and controlled tool orchestration.

The project differentiates itself primarily as an engineering portfolio system: measurable retrieval, explicit authorization, observable agent execution and deliberate reliability design.

6. Differentiation / Value Proposition

- Hybrid retrieval: dense vector search + lexical/BM25 search.
- Reranking to improve the relevance of retrieved chunks.
- Citations tied to source documents/chunks.
- Agentic tool use with strict schemas and authorization.
- Asynchronous document processing with retries and dead-letter handling.
- Redis caching for appropriate repeated queries.
- RAG and agent evaluation rather than relying only on subjective demos.
- Audit logs for sensitive actions and tool execution.
- Observability for API, retrieval, LLM, queue and tool latency.

7. Scope

MVP

- User authentication and organization-aware access.
- PDF/Markdown upload.
- Document validation, parsing and chunking.
- Embedding generation and PostgreSQL/pgvector storage.
- Hybrid lexical + vector retrieval.
- Reranking.
- LLM answer generation with citations.
- Conversation UI.
- Basic document administration and processing status.

Later scope

- Agent/tool orchestration.
- Employee database tool.
- Calendar/holiday tool.
- Async queue and worker scaling.
- Redis caching.

- OpenTelemetry tracing and metrics.
- Load testing and production hardening.

Out of scope initially

- Kubernetes
- Many microservices
- Complex multi-region deployment
- Unnecessary cloud infrastructure
- Fully autonomous high-risk actions

8. Functional Requirements

ID	Requirement
FR-01	Authorized users can upload supported documents.
FR-02	The system validates and stores document metadata.
FR-03	Documents are parsed, cleaned, chunked and indexed.
FR-04	Users can search indexed knowledge.
FR-05	The system performs hybrid retrieval and reranking.
FR-06	The LLM generates answers grounded in retrieved context.
FR-07	Answers expose source citations.
FR-08	Authorized users can invoke approved tools.
FR-09	Tool inputs and outputs are schema-validated.
FR-10	Administrators can inspect processing status and failures.
FR-11	Important operations are recorded in audit logs.

9. Non-Functional Requirements

- **Performance:** define p50/p95 latency targets after establishing a baseline workload.
- **Reliability:** accepted asynchronous jobs must not silently disappear; failed jobs require retry and terminal failure states.
- **Scalability:** retrieval and workers should be independently scalable where justified.
- **Security:** authentication, RBAC, tenant isolation, secret management and least-privilege tool access.
- **AI reliability:** grounded context, citation validation, structured tool calls, step limits and timeouts.
- **Observability:** logs, metrics and traces with correlation/request IDs.
- **Maintainability:** modular services, tests, clear interfaces and documentation.

10. System Architecture

High-level flow:

User → React/Next.js → FastAPI → Authentication/Authorization → Query Service → Hybrid Retrieval → Reranker → Context Builder → LLM → Citation Validation → Response

Document flow:

Upload → Validation → Object/File Storage → Job Creation → Queue → Worker → Parse → Clean → Chunk → Embed → Index → Completed

Advanced agent flow:

User Request → Agent → Plan/Decision → Tool Selection → Schema Validation → Authorization → Tool Execution → Result Validation → Final Response

11. Recommended Tech Stack

Layer	Choice	Reason
Backend	Python + FastAPI	Fast development, strong AI ecosystem, typed APIs.
Frontend	React / Next.js	Modern web UI and existing familiarity.
Database	PostgreSQL	Reliable relational core and strong ecosystem.
Vector search	pgvector	Keeps initial architecture simple.
Lexical search	PostgreSQL full-text/BM25-compatible app	Supports exact-term retrieval.
Cache	Redis	Fast cache and coordination primitive.
Queue	Celery/RQ initially	Simple asynchronous worker model.
LLM	LLM API	High-quality generation and tool calling.
Validation	Pydantic / JSON Schema	Strict data and tool contracts.
Testing	Pytest	Python unit/integration testing.
DevOps	Docker + GitHub Actions	Reproducible development and CI.
Observability	OpenTelemetry + Prometheus/Grafana	Metrics/traces for production learning.

Kafka/Qdrant/Kubernetes should be introduced only when requirements justify them. The project should demonstrate engineering judgment, not technology collection.

12. RAG Architecture

- Parse documents into clean text.
- Split content into meaningful chunks with metadata.
- Generate embeddings.
- Store vectors and metadata in PostgreSQL/pgvector.
- Perform lexical search for exact terminology and identifiers.
- Perform vector search for semantic similarity.
- Fuse candidate results.
- Rerank candidates using a stronger relevance model.
- Build a constrained context window.
- Ask the LLM to answer only from supported context.
- Validate and display citations.

RAG Evaluation

- Retrieval recall / hit rate.
- Context relevance.
- Answer correctness.
- Citation correctness.
- Unsupported-claim / hallucination rate.
- Latency and token/cost measurements.

13. Agent Architecture

The agent should not receive unrestricted access to the system. It operates through a small registry of approved tools.

- Tool definitions use strict input/output schemas.
- Authorization is checked before execution.
- Tool calls have timeouts and maximum execution/step limits.
- External results are treated as untrusted input.
- Sensitive operations require explicit permission and potentially user confirmation.
- Every tool invocation can be logged for audit and debugging.

14. Database Design

Initial entities:

- Organization(id, name, created_at)
- User(id, organization_id, email, role_id, created_at)
- Role(id, name, permissions)
- Document(id, organization_id, name, type, status, storage_ref, created_at)
- DocumentChunk(id, document_id, chunk_index, text, embedding, metadata)
- Conversation(id, user_id, created_at)
- Message(id, conversation_id, role, content, created_at)
- Tool(id, name, schema, enabled)
- ToolExecution(id, tool_id, user_id, input, output, status, latency, created_at)
- Job(id, type, status, attempts, error, created_at, updated_at)
- AuditLog(id, organization_id, user_id, action, resource, metadata, created_at)

Indexes should be designed around actual query patterns. Important candidates include organization_id, document_id, status, timestamps and vector indexes. Access-control predicates must be incorporated into retrieval so users cannot retrieve another organization's documents.

15. API Design

- POST /auth/login
- POST /documents
- GET /documents
- GET /documents/{id}
- POST /documents/{id}/reprocess
- GET /jobs/{id}
- POST /chat
- GET /conversations/{id}
- GET /tools
- POST /tools/{id}/execute

- GET /admin/jobs
- GET /admin/audit-logs

Exact endpoint design should be refined after user stories and UI flows are finalized.

16. UI/UX

Main screens

- Login / authentication.
- Dashboard: recent conversations, indexed documents, processing status and system summary.
- Chat: question, answer, citations and tool activity where appropriate.
- Documents: upload, status, source metadata, retry/reprocess.
- Admin: users, roles, failed jobs and audit information.

UX states

- Loading
- Empty
- Processing
- Success
- Partial result
- Failure/retry
- Unauthorized
- No relevant knowledge found

The interface should make uncertainty visible. If the system lacks sufficient evidence, it should say so rather than presenting a confident unsupported answer.

17. Key Workflows

Question Answering

Question → Auth → Query processing → Hybrid retrieval → Reranking → Context → LLM → Citation validation → Answer

Document Ingestion

Upload → Validate → Store → Queue → Worker → Parse → Chunk → Embed → Index → Complete

Failure Handling

Worker failure → Retry with backoff → Retry limit → Dead-letter/failed state → Admin visibility → Reprocess

18. Security & Privacy

- Authentication and session/token security.
- Role-based access control.
- Organization/tenant isolation.
- Least-privilege tool permissions.

- Secure secret storage and environment configuration.
- Encryption in transit and appropriate encryption at rest.
- Audit logs for sensitive operations.
- Prompt-injection defenses and untrusted-document handling.
- File validation and resource limits.
- LLM provider data-retention/privacy review before production use.

19. Testing Strategy

- **Unit tests:** parsers, chunkers, retrieval fusion, authorization and validators.
- **Integration tests:** PostgreSQL/pgvector, Redis, queue and LLM adapters.
- **API tests:** authentication, permissions, validation and error responses.
- **RAG evaluation:** curated question/answer/source datasets.
- **Agent tests:** tool selection, schema failures, unauthorized actions, timeouts and retries.
- **Load tests:** concurrent queries, queue throughput and database performance.
- **E2E:** upload → processing → chat → citation flow.

20. Observability

- Structured application logs.
- Request/correlation IDs.
- API latency and error rate.
- Retrieval latency and result counts.
- LLM latency, token usage and cost.
- Cache hit/miss rate.
- Queue depth, processing time and retry count.
- Agent step count and tool latency.
- Distributed traces using OpenTelemetry.

21. Caching Strategy

Redis can cache safe, repeatable results such as selected retrieval results or final responses when permissions and data freshness are controlled.

- Define a deterministic cache key.
- Include organization/user permission context where required.
- Set TTL based on data freshness.
- Invalidate when source documents change.
- Measure hit rate before optimizing further.

22. Distributed Processing & Reliability

Document ingestion should be asynchronous because parsing and embedding large documents can take significant time.

- API creates a durable job record.
- Queue delivers work to workers.
- Workers process jobs independently.
- Transient failures retry with exponential backoff.
- Jobs become terminally failed after a bounded retry count.
- Idempotency prevents duplicate processing from corrupting state.
- Dead-letter/failed jobs remain visible for investigation.
- Only add Kafka when throughput or event-stream requirements justify its operational cost.

23. Agile Development Plan

Sprint	Goal
0	Vision, requirements, scope, user stories, architecture, database, UI/UX, repo setup.
1	Document upload, validation and storage.
2	Parsing, chunking, embeddings and pgvector.
3	Hybrid retrieval.
4	Reranking + grounded RAG + citations.
5	RAG evaluation and quality improvements.
6	Agent + first safe tools.
7	Async queue + workers + retries.
8	Redis caching and performance measurement.
9	Reliability, idempotency and failure handling.
10	Observability and dashboards.
11	Security hardening and testing.
12	Deployment, documentation and production-style release.

Each sprint follows: Sprint Planning → Development → Testing → Review/Demo → Retrospective → Backlog adjustment.

24. Project Repository / Documentation Structure

```
docs/ ■■■ 01-project-overview.md ■■■ 02-requirements.md ■■■ 03-product-scope.md ■■■ 04-user-stories.md
■■■ 05-ui-ux.md ■■■ 06-system-architecture.md ■■■ 07-rag-architecture.md ■■■ 08-agent-architecture.md
■■■ 09-database-design.md ■■■ 10-api-design.md ■■■ 11-workflows.md ■■■ 12-security.md ■■■
13-testing.md ■■■ 14-observability.md ■■■ 15-deployment.md ■■■ 16-risks-tradeoffs.md ■■■
17-references.md
```

25. Risks & Trade-offs

- Hallucination → grounding, citations and evaluation.
- Poor retrieval → hybrid search, chunking experiments and reranking.
- High LLM cost → caching, smaller models where appropriate and token controls.
- Slow ingestion → asynchronous workers and batching.

- Database bottlenecks → indexes, query profiling and measured scaling.
- Agent misuse → strict tools, authorization, schemas and step limits.
- Vendor lock-in → adapter interfaces and documented provider assumptions.
- Overengineering → introduce infrastructure only when a requirement demonstrates its need.

26. References & Learning Resources

The following technologies and concepts are the primary reference areas for implementation. URLs can be added/verified during the research phase before the project begins.

- FastAPI documentation — API design, validation and dependency injection.
- PostgreSQL documentation — relational data, indexing and full-text search.
- pgvector documentation — vector storage and similarity search in PostgreSQL.
- Redis documentation — caching and data structures.
- Pydantic documentation — typed validation and schemas.
- Pytest documentation — automated testing.
- OpenTelemetry documentation — traces, metrics and instrumentation.
- Docker documentation — containerization.
- GitHub Actions documentation — CI/CD.
- Relevant LLM provider documentation — structured outputs and tool/function calling.
- RAG evaluation literature and benchmark methodology — retrieval and grounded generation quality.

27. Engineering Learning Goals

- Design APIs and system boundaries instead of only writing endpoints.
- Understand search and information retrieval.
- Measure AI quality rather than trusting demos.
- Build reliable asynchronous systems.
- Learn caching, idempotency, retries and failure modes.
- Implement authorization and secure tool execution.
- Use tests, metrics, logs and traces to debug systems.
- Make architecture decisions based on requirements and trade-offs.
- Use AI coding assistants productively while retaining ownership of engineering decisions.

28. Definition of Done

- Feature has clear acceptance criteria.
- Implementation is tested.
- Relevant error paths are handled.
- Security/authorization is considered.
- Logs/metrics are added where useful.
- Documentation is updated.

- Code is committed through Git with a meaningful message.
- Feature is demonstrated in the sprint review.
- Known limitations are recorded rather than hidden.

29. First Step — Sprint 0

Do not start by coding the RAG engine. First freeze the product scope and define one primary user persona.

Recommended immediate sequence:

- Choose the MVP persona/use case.
- Write 5–10 concrete user stories.
- Define MVP and out-of-scope features.
- Set initial measurable NFR targets.
- Finalize the first architecture diagram.
- Design the database ER model.
- Create the first UI wireframes.
- Create the Git repository and documentation structure.
- Then begin Sprint 1.

Document status: Planning baseline — intended to evolve through Agile sprint reviews and engineering evidence.